



University
of Glasgow

**WORLD
CHANGING
GLASGOW**

Embedded System Testing, and Debugging - Part 2

Dr Sajjad Hussain

Associate Professor

UESTC 2004: Embedded Processors

Sildes by: Dr Yao Sun





Aim

- Understand Task Scheduling in Embedded Systems
- Testing & Debugging - Interrupt

Learning Objectives

By the end of this lesson you will be able to:

- Understand the meaning of multitasking in embedded systems.
- Explain the differences between different scheduling algorithms.
- An example of interrupt coding

Assumed Previous Knowledge

- Micro Electronics



- Understand Task Scheduling in Embedded Systems
- Testing & Debugging - Interrupt



- Understand Task Scheduling in Embedded Systems
- Testing & Debugging - Interrupt

Preemptive vs. Nonpreemptive Scheduling

- **Preemptive processes**
 - Can be removed from their current processor
 - Can lead to improved response times
 - Important for interactive environments
 - Preempted processes remain in memory
- **Nonpreemptive processes**
 - Run until completion or until they yield control of a processor
 - Unimportant processes can block important ones indefinitely

15.2.2 Preemptive Scheduling

In a **preemptive scheduling** once the CPU is given to a task it can be taken away, for example when a higher priority task wants the CPU. Preemptive scheduling is used in real-time systems where the tasks are usually configured with different priorities and time critical tasks are given higher priorities. A higher priority task can stop a lower priority one and grab and use the CPU until it releases it. In preemptive scheduling the task contexts are saved so that the tasks can resume their operations from the point they left when they are given back the CPU.

Non-preemptive scheduling – First Come First Served (FCFS)/First In First Out (FIFO) Scheduling:

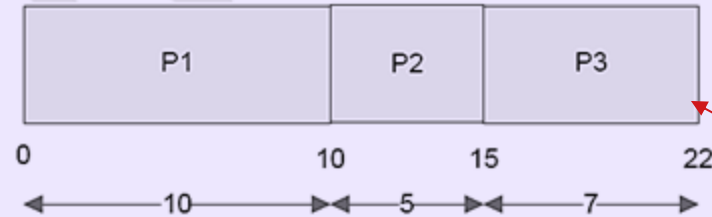
- Allocates CPU time to the processes based on the order in which they enters the '*Ready*' queue
- The first entered process is serviced first
- It is same as any real world application where queue systems are used; E.g. Ticketing

Drawbacks:

- Favors monopoly of process. A process, which does not contain any I/O operation, continues its execution until it finishes its task
- In general, FCFS favors CPU bound processes and I/O bound processes may have to wait until the completion of CPU bound process, if the currently executing process is a CPU bound process. This leads to poor device utilization.
- The average waiting time is not minimal for FCFS scheduling algorithm

EXAMPLE: Three processes with process IDs P1, P2, P3 with estimated completion time 10, 5, 7 milliseconds respectively enters the ready queue together in the order P1, P2, P3. Calculate the waiting time and Turn Around Time (TAT) for each process and the Average waiting time and Turn Around Time (Assuming there is no I/O waiting for the processes).

Solution: The sequence of execution of the processes by the CPU is represented as



Consider as non-preemptive, FCFS

Gantt Chart

Burst Time

Embedded Firmware – Non-preemptive

Waiting Time for P1 = 0 ms (P1 starts executing first)

Waiting Time for P2 = 10 ms (P2 starts executing after completing P1)

Waiting Time for P3 = 15 ms (P3 starts executing after completing P1 and P2)

Average waiting time = (Waiting time for all processes) / No. of Processes

$$= (\text{Waiting time for (P1+P2+P3)}) / 3$$

$$= (0+10+15)/3 = 25/3 = 8.33 \text{ milliseconds}$$

Turn Around Time (TAT) for P1 = 10 ms (Time spent in Ready Queue +
Execution Time)

Turn Around Time (TAT) for P2 = 15 ms (-Do-)

Turn Around Time (TAT) for P3 = 22 ms (-Do-)

Average Turn Around Time = (Turn Around Time for all processes) / No. of
Processes

$$= (\text{Turn Around Time for (P1+P2+P3)}) / 3$$

$$= (10+15+22)/3 = 47/3$$

$$= 15.66 \text{ milliseconds}$$

Non-preemptive scheduling – Last Come First Served (LCFS)/Last In First Out (LIFO) Scheduling:

- Allocates CPU time to the processes based on the order in which they are entered in the '*Ready*' queue
- The last entered process is serviced first

Drawbacks:

- Favors monopoly of process. A process, which does not contain any I/O operation, continues its execution until it finishes its task
- In general, LCFS favors CPU bound processes and I/O bound processes may have to wait until the completion of CPU bound process, if the currently executing process is a CPU bound process. This leads to poor device utilization.
- The average waiting time is not minimal for LCFS scheduling algorithm

Non-preemptive scheduling – Shortest Job First (SJF) Scheduling.

- Allocates CPU time to the processes based on the execution completion time for tasks
- The average waiting time for a given set of processes is minimal in SJF scheduling
- Optimal compared to other non-preemptive scheduling like FCFS

Drawbacks:

- A process whose estimated execution completion time is high may not get a chance to execute if more and more processes with least estimated execution time enters the 'Ready' queue before the process with longest estimated execution time starts its execution
- May lead to the 'Starvation' of processes with high estimated completion time
- Difficult to know in advance the next shortest process in the 'Ready' queue for scheduling since new processes with different estimated execution time keep entering the 'Ready' queue at any point of time.

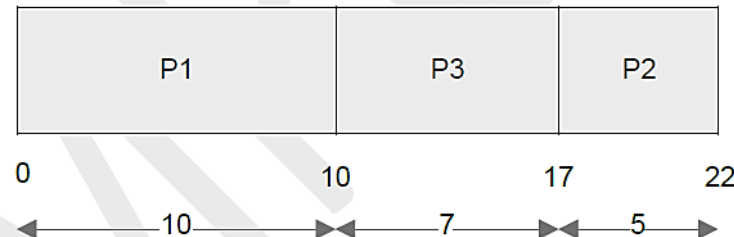
Non-preemptive scheduling – Priority based Scheduling

- A priority, which is unique or same is associated with each task
- The priority of a task is expressed in different ways, like a priority number, the time required to complete the execution etc.
- In number based priority assignment the priority is a number ranging from 0 to the maximum priority supported by the OS. The maximum level of priority is OS dependent.
- Windows CE supports 256 levels of priority (0 to 255 priority numbers, with 0 being the highest priority)

- The non-preemptive priority based scheduler sorts the 'Ready' queue based on the priority and picks the process with the highest level of priority for execution

EXAMPLE: Three processes with process IDs P1, P2, P3 with estimated completion time 10, 5, 7 milliseconds and priorities 0, 3, 2 (0- highest priority, 3 lowest priority) respectively enters the ready queue together. Calculate the waiting time and Turn Around Time (TAT) for each process and the Average waiting time and Turn Around Time (Assuming there is no I/O waiting for the processes) in priority based scheduling algorithm.

Solution: The scheduler sorts the '*Ready*' queue based on the priority and schedules the process with the highest priority (P1 with priority number 0) first and the next high priority process (P3 with priority number 2) as second and so on. The order in which the processes are scheduled for execution is represented as



The waiting time for all the processes are given as

Waiting Time for P1 = 0 ms (P1 starts executing first)

Waiting Time for P3 = 10 ms (P3 starts executing after completing P1)

Waiting Time for P2 = 17 ms (P2 starts executing after completing P1 and P3)

Average waiting time = (Waiting time for all processes) / No. of Processes

$$= (\text{Waiting time for (P1+P3+P2)}) / 3$$

$$= (0+10+17)/3 = 27/3$$

$$= 9 \text{ milliseconds}$$

Turn Around Time (TAT) for P1 = 10 ms (Time spent in Ready Queue + Execution Time)

Turn Around Time (TAT) for P3 = 17 ms (-Do-)

Turn Around Time (TAT) for P2 = 22 ms (-Do-)

Average Turn Around Time = (Turn Around Time for all processes) / No. of Processes

$$= (\text{Turn Around Time for (P1+P3+P2)}) / 3$$

$$= (10+17+22)/3 = 49/3$$

$$= 16.33 \text{ milliseconds}$$



Drawbacks:

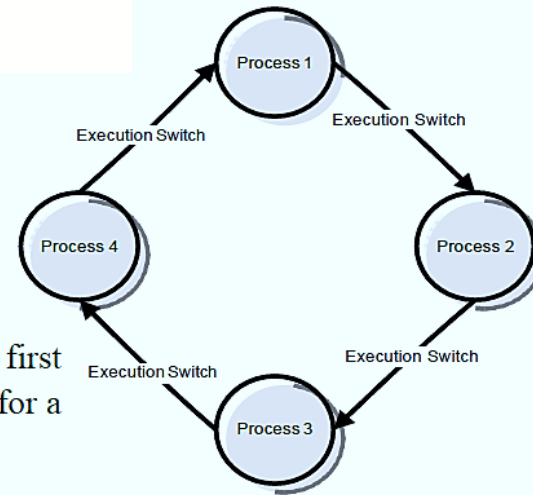
- Similar to SJF scheduling algorithm, non-preemptive priority based algorithm also possess the drawback of '*Starvation*' where a process whose priority is low may not get a chance to execute if more and more processes with higher priorities enter the '*Ready*' queue before the process with lower priority starts its execution.

Preemptive scheduling – Preemptive SJF Scheduling/ Shortest Remaining Time (SRT):

- The *non preemptive SJF* scheduling algorithm sorts the 'Ready' queue only after the current process completes execution or enters wait state, whereas the *preemptive SJF* scheduling algorithm sorts the 'Ready' queue when a new process enters the 'Ready' queue and checks whether the execution time of the new process is shorter than the remaining of the total estimated execution time of the currently executing process
- If the execution time of the new process is less, the currently executing process is preempted and the new process is scheduled for execution

Preemptive scheduling – Round Robin (RR) Scheduling:

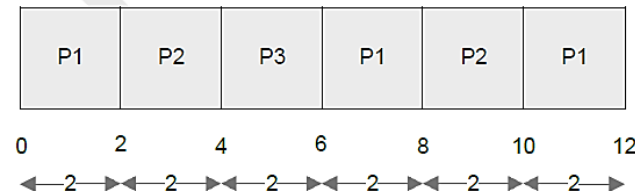
- Each process in the 'Ready' queue is executed for a pre-defined time slot.
- The execution starts with picking up the first process in the 'Ready' queue. It is executed for a pre-defined time



- When the pre-defined time elapses or the process completes (before the pre-defined time slice), the next process in the 'Ready' queue is selected for execution.
- This is repeated for all the processes in the 'Ready' queue
- Once each process in the 'Ready' queue is executed for the pre-defined time period, the scheduler comes back and picks the first process in the 'Ready' queue again for execution.
- Round Robin scheduling is similar to the FCFS scheduling and the only difference is that a time slice based preemption is added to switch the execution between the processes in the 'Ready' queue

EXAMPLE: Three processes with process IDs P1, P2, P3 with estimated completion time 6, 4, 2 milliseconds respectively, enters the ready queue together in the order P1, P2, P3. Calculate the waiting time and Turn Around Time (TAT) for each process and the Average waiting time and Turn Around Time (Assuming there is no I/O waiting for the processes) in RR algorithm with Time slice= 2ms.

Solution: The scheduler sorts the 'Ready' queue based on the FCFS policy and picks up the first process P1 from the 'Ready' queue and executes it for the time slice 2ms. When the time slice is expired, P1 is preempted and P2 is scheduled for execution. The Time slice expires after 2ms of execution of P2. Now P2 is preempted and P3 is picked up for execution. P3 completes its execution within the time slice and the scheduler picks P1 again for execution for the next time slice. This procedure is repeated till all the processes are serviced. The order in which the processes are scheduled for execution is represented as



Waiting Time for P1 = $0 + (6-2) + (10-8) = 0+4+2 = 6\text{ms}$ (P1 starts executing first and waits for two time slices to get execution back and again 1 time slice for getting CPU time)

Waiting Time for P2 = $(2-0) + (8-4) = 2+4 = 6\text{ms}$ (P2 starts executing after P1 executes for 1 time slice and waits for two time slices to get the CPU time)

Waiting Time for P3 = $(4 - 0) = 4\text{ms}$ (P3 starts executing after completing the first time slices for P1 and P2 and completes its execution in a single time slice.)

Average waiting time = (Waiting time for all the processes) / No. of Processes

$$= (\text{Waiting time for (P1+P2+P3)}) / 3$$

$$= (6+6+4)/3 = 16/3$$

$$= 5.33 \text{ milliseconds}$$

Turn Around Time (TAT) for P1 = 12 ms (Time spent in Ready Queue + Execution Time)

Turn Around Time (TAT) for P2 = 10 ms (-Do-)

Turn Around Time (TAT) for P3 = 6 ms (-Do-)

Average Turn Around Time = (Turn Around Time for all the processes) / No. of Processes

= (Turn Around Time for (P1+P2+P3)) / 3

= (12+10+6)/3 = 28/3

= 9.33 milliseconds.

Embedded Firmware – Round Robin Preemptive Scheduling

SERIAL NO.	ADVANTAGES	DISADVANTAGES
1.	There is fairness since every process gets equal share of CPU.	There is Larger waiting time and Response time.
2.	The newly created process is added to end of ready queue.	There is Low throughput.
3.	A round-robin scheduler generally employs time-sharing, giving each job a time slot or quantum.	There is Context Switches.
4.	While performing a round-robin scheduling,a particular time quantum is allotted to different jobs.	Gantt chart seems to come too big (if quantum time is less for scheduling.For Example:1 ms for big scheduling.)
5.	Each process get a chance to reschedule after a particular quantum time in this scheduling.	Time consuming scheduling for small quantumms .

Preemptive scheduling – Priority based Scheduling

- Same as that of the *non-preemptive priority* based scheduling except for the switching of execution between tasks
- In *preemptive priority* based scheduling, any high priority process entering the 'Ready' queue is immediately scheduled for execution whereas in the *non-preemptive* scheduling any high priority process entering the 'Ready' queue is scheduled only after the currently executing process completes its execution or only when it voluntarily releases the CPU
- The priority of a task/process in preemptive priority based scheduling is indicated in the same way as that of the mechanisms adopted for non-preemptive multitasking.



- Understand Task Scheduling in Embedded Systems
- Testing & Debugging - Interrupt

How to handle the events generated by the devices while CPU is busy in **executing another process**?

Polling and **Interrupt** are the two ways

What is polling?

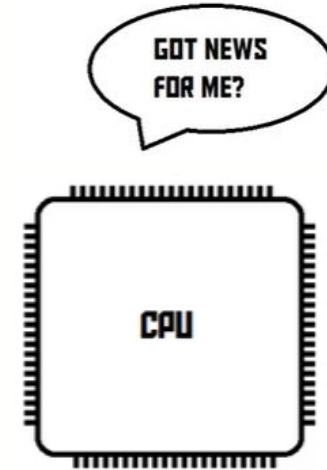
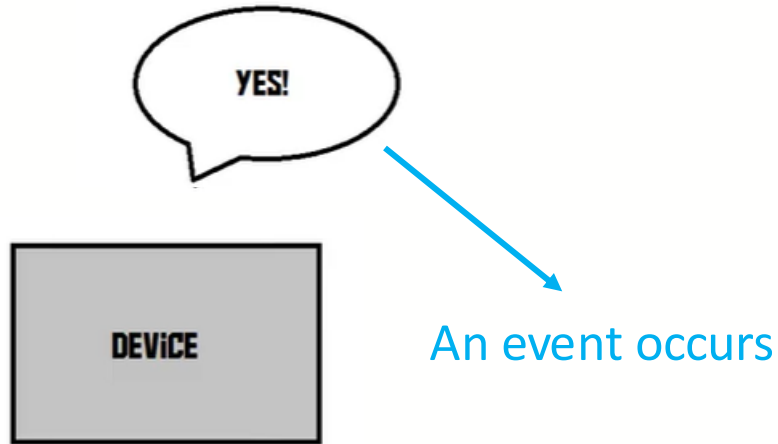
Polling is the protocol that indicates the **CPU** that a device requires its attention. It is a **continuous act** to figure out whether the device is working properly.

Why needs polling?

In this process, the **CPU continuously checks** the status of the device to see if it needs to be taken care of.

Protocols are the means by which **a CPU services a device.**

Polling - Concept



Interrupt - Concept

- **Definition:** An **interrupt** is the **automatic transfer** of software **execution** in response to a hardware **event** that is asynchronous with the current software execution.
- **Examples:**
 - Mouse moved
 - Keyboard key pressed
 - Printer out of paper
 -

Interrupt - Concept

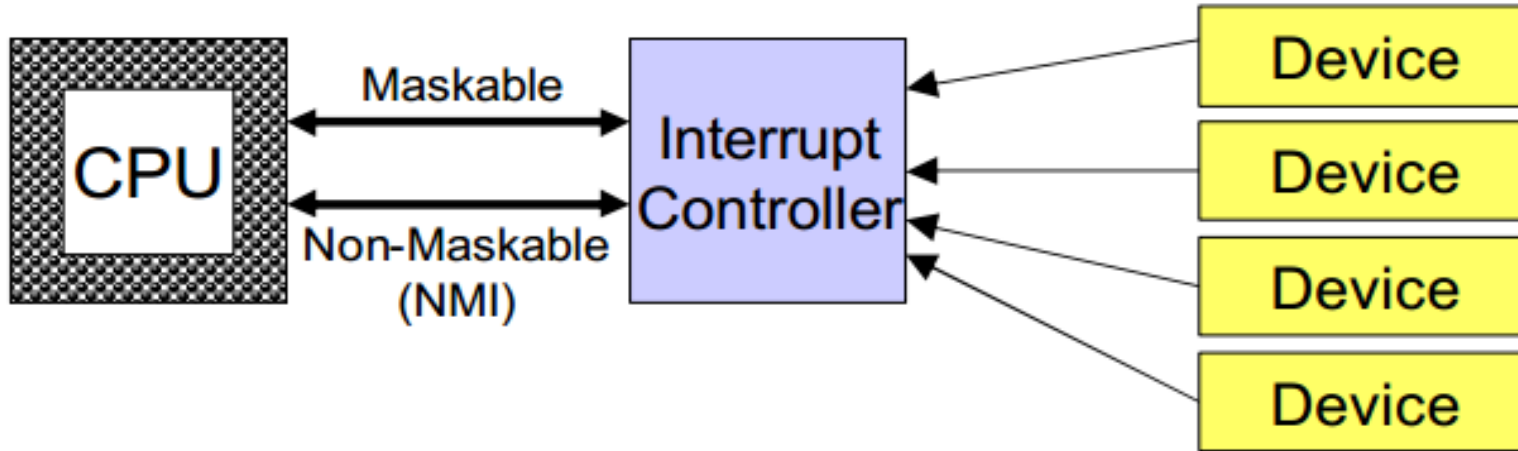


Figure Interrupt

Interrupt - Concept

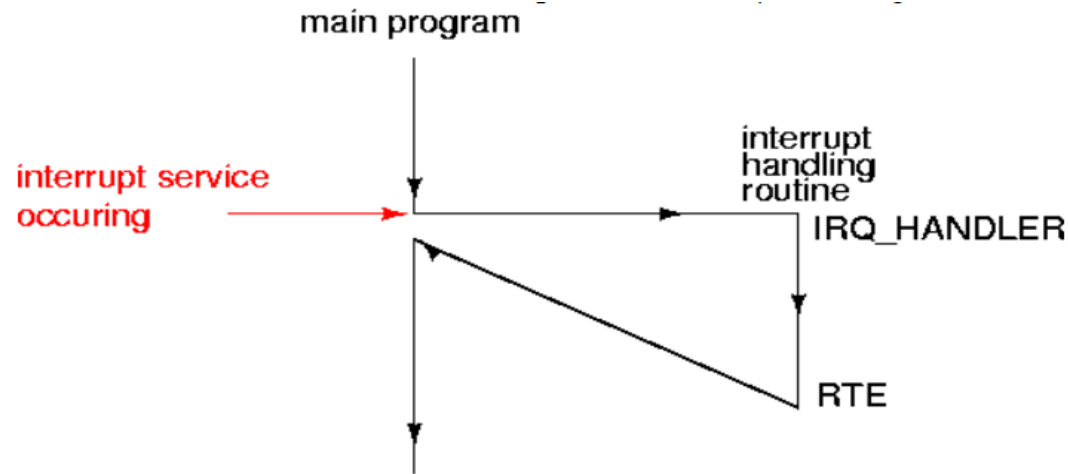


Figure Interrupt Servicing.

- **Response to an interrupt**
 - 1) Test interrupts occur (more than one interrupts)
 - 2) **Current instruction is stopped,**
 - 3) **Protect the status** of current instruction, Eight registers are pushed on the stack,
 - 4) LR is set to 0xFFFFFFFF9,
 - 5) IPSR is set to the **interrupt number,**
 - 6) PC is loaded with the interrupt vector

- **Response to an interrupt (cont'd)**
 -
 - 7) Execute interrupt instruction/program,
 - 8) Get the interrupt return instruction,
 - 9) Go back to the main program (where to go?)



Quiz

When the interrupt program is finished, **which line** should the main program return to?

Quiz

When the interrupt program is finished, **which line** should the main program return to?

Answer

- The line that the interrupt occurred, i.e., current instruction;
- The next line that the interrupt occurred, i.e., next instruction ;
(read a data; write a data ..)
- Abort (there is an error)



Preparation:

1. C language Compiler
Dev C++; **Visual Studio**;
2. Basic knowledge of C
3. Process of dealing with an interrupt

Case Study

Let us consider a main process with an interrupt, where the **main process** is to input 10 random numbers with range **[0, 100]**, and the **interrupt** is to input $\{11^2, 12^2, 13^2, \dots, 20^2\}$ **once** the input number in the main process is **out of range**.

Let us implement this program in C!



Case Study

Main Process: Input 10 numbers in range $[0, 100]$;

Interrupt Process: Input $\{11^2, 12^2, 13^2, \dots, 20^2\}$;

Trigger Condition: The number is out of range in the main process.

Interrupt – Design (C)

Case Study

```
12;  
20;  
70;  
120;  
52;  
41;  
33;  
14;  
22;  
39;  
70;
```

Interrupt:

$\{11^2, 12^2, 13^2, \dots, 20^2\}$

Output:

12;20;70;52;41;33;14;22;39;70

The **same order** with the input in the main process
without 120.

Case Study

Headers

```
#include<iostream>
#include<vector>
#include <Windows.h>
using namespace std;
```

```
extern vector<int> Register(10, 0); //simulate register to store the current data in main process
extern vector<int> Stack(10, 0); //simulate stack to protect status
```

two vectors to store data in Register and Stack

Case Study

Protect Function

```
/*Protect status. Put the data from Register to Stack*/  
void Protect()  
{  
    Sleep(2000); //2 seconds delayed  
    cout << "Start to protect status ! \n";  
    for (int m = 0; m < 10; m++)  
    {  
        Stack[m] = Register[m];  
    }  
    Sleep(2000);  
    cout << "Finish the protect process ! \n";  
}
```

The data is protected in Stack

Interrupt – Design (C)

Case Study

Recover Function

```
/*Recover status of main process, Put data from Stack to Register*/  
void Recover()  
{  
    Sleep(2000);  
    cout << "Begin to recover status ! \n";  
    for (int n = 0; n < 10; n++)  
    {  
        Register[n] = Stack[n];  
        Stack[n] = 0;  
    }  
    Sleep(2000);  
    cout << "Finish the recover process ! \n";  
}
```

The data is recovered from Stack

Case Study

Main Function

```
/*main program*/
int main(void)
{
    int Flag = 0; //interrupt indicator. 1: switch on, 0: switch off.
    char Choice[] = "N"; //decide to switch on/off interrupt

    cout << "Please indicate whether to switch on interrupt (Y: ON, N: OFF): \n";
    cin >> Choice;

    if (strcmp(Choice, "Y") == 0)
    {
        Flag = 1;
    }
}
```

Case Study

Main Function (cont'd)

```
for (int i = 0; i < 10; i++)  
{  
    cout << "Please insert number" " " << i + 1 << ":" << endl;  
    cin >> tem;  
    cout << "\n";  
  
    if ((tem > 100) || (tem < 0))  
    {  
        cout << "Out of range, interrupt is triggered! \n";  
        if (Flag == 1)
```

Interrupt occurs

Case Study

Main Function (cont'd)

Response to interrupt



```
if (Flag == 1)
{
    Protect();
    Sleep(2000);
    cout << "Execute interrupt program! \n";
    Interrupt();
    Sleep(2000);
    cout << "Interrupt program is finished! \n";
    Recover();
    i = i - 1;
    continue;
}
else
{
    Sleep(1000);
    cout << "Interrupt is switched off! \n";
}
```

Case Study

Main Function (cont'd)

Output the results of **main program**

```
//output the 10 numbers
cout << "The input 10 numbers are : \n";
for (int j = 0; j < 10; j++)
{
    cout << Register[j] << "\t";
}
// return 0;
```




Case Study

- Now let us see how this program can be run and debugged in C language compiler.
- We use Microsoft Visual Studio as an example.



University
of Glasgow



WORLD
CHANGING
GLASGOW

Summary

- Understand multitasking in real-time embedded systems.
- Several scheduling algorithms: **preemptive** and **non-preemptive**.
- Discussed **priority-based** and **non-priority-based** scheduling algorithms.

